

Digital Circuits and VHDL – Overview and Examples

Florian Hagemann

Abstract—Summary on digital circuits and VHDL. Starting with basic switches and then transitioning to transistors, logic gates, boolean algebra. Then we look at VHDL syntax basics and move from combinatorial circuits and basic sequential circuits to moore and mealy finite state machines.

CONTENTS

I	Introduction to Digital Circuits	1
I-A	Electronic Switches: Transistors	2
I-B	Boolean Algebra	2
I-C	Logic Gates	2
II	Introduction to VHDL	3
II-A	Hardware Description Languages	3
II-B	Basic Syntax	3
II-C	Blackbox Diagrams and Entities	4
III	Combinational Circuits	5
IV	Sequential Circuits	5
V	Finite State Machines (FSM)	5
V-A	Moore Machines	5
V-B	Mealy Machines	6
	References	7

I. INTRODUCTION TO DIGITAL CIRCUITS

WHAT is a *digital circuit*? To understand this, we need to examine each word separately. A *circuit* is a closed loop that allows electricity to flow. Typically it consists of a power source and some device (e.g., a motor, LED, etc.) that uses that electricity. A circuit needs to be closed in order for the electricity to flow through it [1]. *Digital* comes from the Latin “*digitalis*” meaning finger or toe. Digital can mean “something to do with digits (fingers or toes)” but can also mean the “property of representing values as discrete, often binary, numbers rather than a continuous spectrum” [2].

A **digital circuit** is a closed loop that electricity can flow through that uses discrete values. Most often these values are ‘1’ (on, high, or true) and ‘0’ (off, low, or false)¹. This system of using two values is called *binary*.

We can now look at a very basic example (Fig. 1) of what we can do by defining binary values. When the switch is closed, electricity can flow through the LED and it will light up; if it is not closed, no electricity will flow.

¹You can use more than two values but binary is most common. See https://en.wikipedia.org/wiki/Ternary_computer

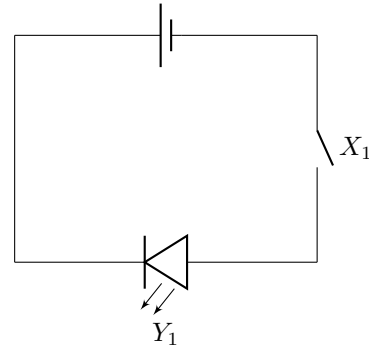


Fig. 1. Simple series circuit with a voltage source, switch, and LED.

We only care for the switch and the output state of the circuit though, not what the power source is or what the output is connected to, so we can simplify our diagram (Fig. 2).

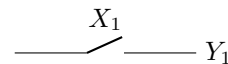


Fig. 2. Simplified diagram of Fig. 1.

So we know if we press X_1 , Y_1 will be set high or to ‘1’. We can illustrate this using a *truth table* (Tab. I):

TABLE I
TRUTH TABLE FOR FIG. 2

X_1	Y_1
0	0
1	1

With a truth table we can read what our output will be for any input. At this point in time this is overkill, but it gets more useful the more switches we add. For example, we can add another switch in series (Fig. 3).

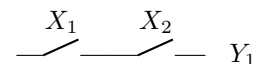


Fig. 3. Two switches in series

Now if none or only one switch is turned on, the output will stay low. But if both switches are turned on, the output will be high. So we can check a condition just using switches and electricity. But we might also notice that if we want to do something more complex, we will need more and more switches which we have to flip manually.

TABLE II
TRUTH TABLE FOR FIG. 3

X_1	X_2	Y_1
0	0	0
1	0	0
0	1	0
1	1	1

A. Electronic Switches: Transistors

Luckily there is an invention that solves the issue of us having to flip switches by hand. The **transistor** is a semiconductor device used (among other uses) to switch electrical signals². Transistors usually have three terminals a *collector*, *base*, and *emitter*. You might have heard a water analogy used to explain electricity before and we can use it again to understand the transistor. The water flow comes from the source, applying a voltage to the gate opens it, and the water drains through the drain. If we don't open the gate, no water can flow and the drain will be dry. There are many types of transistors, but we can't talk about all of them, but in general they do the same thing.

We can now replace our switches in Figure 3 with transistors, we will look at the circuit as a whole though (Fig. 4). Now we only need to apply a voltage to X_1 and X_2 instead of switching them. While at this scale it will still be the same amount of work for us to switch the voltages, the advantage of this design is that we can now plug the output of these transistors into another transistor to switch that automatically.

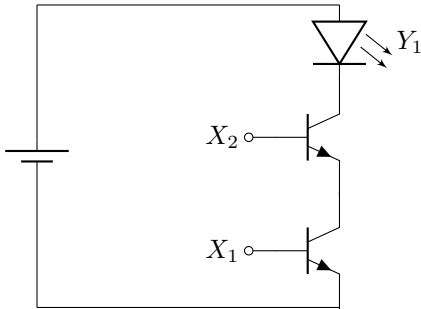


Fig. 4. Simple transistor switch circuit controlling an LED output (Y_1) via input signals X_1 and X_2 .

We represent transistors (here NPN-Transistors) using this symbol (Fig 5).

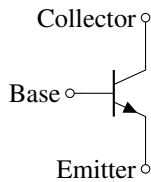


Fig. 5. NPN-Transistor

²We will not talk about how a transistor works. See <https://en.wikipedia.org/wiki/Transistor> or search the web for more information.

B. Boolean Algebra

We've been treating the binary system mostly as "on" and "off" up to this point, but in the beginning we also defined '0' as *false* and '1' as *true*. We can use mathematical formula to describe our circuits. For example we can describe the circuit in Figure 3 with the formula

$$X_1 \wedge X_2 = Y_1$$

. The $\wedge$ symbol means logical AND with the upside down $\vee$ meaning logical OR. How they operate is in the name:

- For $\wedge$ or $\cdot$ (AND): *Input One and Input Two* need to be true for the output to be true.
- For $\vee$ or $+$ (OR): *Input One or Input Two* needs to be true for the output to be true.

We can also look at the truth tables of both logical operations (remember that we represent *false* with 0 and *true* with 1).

TABLE III
TRUTH TABLES FOR AND AND OR OPERATIONS

AND ($X_1 \wedge X_2$)			OR ($X_1 \vee X_2$)		
X_1	X_2	Y_1	X_1	X_2	Y_1
0	0	0	0	0	0
1	0	0	1	0	1
0	1	0	0	1	1
1	1	1	1	1	1

Another operator is *negation*, which is written with the $\neg$ symbol, but can also be written like this

$$\neg X = \overline{X}$$

, which you would read as "not X " (Tab. IV)³.

TABLE IV
TRUTH TABLE FOR NOT

X	$\overline{X}$
0	1
1	0

C. Logic Gates

We want to use these logical operation a lot without drawing a bunch of transistors. That is why there are standard ways of drawing so called **logic gates**. These can be defined as "a device that performs a Boolean function, a logical operation performed on one or more binary inputs that produces a single binary output" [3]. We have already unknowingly used an AND gate in Figure 4. We can redraw our diagram to use a logic gate notation to save ourselves time and space, while making what the circuit does more clear (Fig. 6)⁴.

These logic gates are very simple and can be used to build a lot of other useful circuits. For example we can build a circuit

³Check out K-Maps to simplify your boolean algebra

⁴There are several different notation styles for logic gates. We will use the European notation. See <https://jlcpcb.com/blog/logic-gate-symbols-guide> for an overview of european and ANSI/IEEE equivalents



Fig. 6. AND gate with European notation.

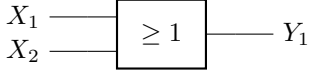


Fig. 7. OR gate with European notation.

that only outputs a '1' if only one of the inputs is '1' (Fig. 9 and tab. V). This kind of circuit is called an XOR gate (XOR being short for exclusive or).

TABLE V
TRUTH TABLE FOR XOR

X_1	X_2	Y_1
0	0	0
1	0	1
0	1	1
1	1	0

The main point of logic gates is that we can abstract the electronical components away to only focus on the logic, but we always need to remember that everything we are doing is just chaining transistor together.

II. INTRODUCTION TO VHDL

A. Hardware Description Languages

A question you might be asking yourself is: How can we build digital circuits? There are many ways of building digital circuits. For example, we could use a breadboard and physically connect transistors, but this gets impossible when you want to build a computer and need billions of transistors that are only a few nanometers big.

We really don't want to even bother with logic gates at all, so we invented **Hardware Description Languages** (HDL) to make digital circuit design easier. You can describe a circuit using an HDL and a *synthesizer* produces the optimized gates for you. As we cover more complex circuits the need for an HDL will get clearer, but we need to start out with the basics to understand how to use an HDL. We will be using *VHDL* as it is one of the two leading HDLs alongside *SystemVerilog*⁵. We will use *GHDL*⁶ as our compiler and simulator, and *GTKWave*⁷ as our waveform viewer. VHDL stands for *VHSIC Hardware Description Language* with VHSIC standing for *Very High Speed Integrated Circuit* (engineers just love acronyms).

⁵We won't cover how to install VHDL or all of its quirks. See *FREE RANGE VHDL* at <https://github.com/fabriziotappero/Free-Range-VHDL-book> for very detailed information

⁶GHDL: <https://ghdl.github.io/ghdl/>

⁷GTKWave: <https://gtkwave.github.io/gtkwave/>

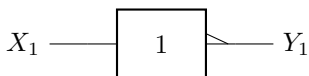


Fig. 8. NOT gate with European notation.

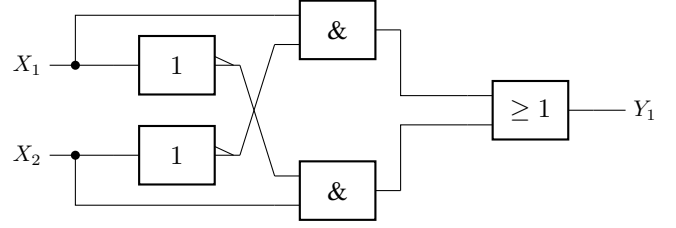


Fig. 9. XOR gate constructed from NOT, AND, and OR gates.

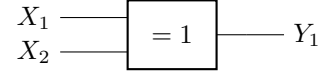


Fig. 10. XOR gate with European notation.

B. Basic Syntax

Some basic features and syntax is needed before we can start using VHDL to describe our circuits⁸

- VHDL is **not** case-sensitive
- VHDL is **not** sensitive to white space
- **Comments** begin with "--" (two consecutive dashes)
- Use of parentheses is recommended to improve readability (For example, in Listing 1 both statements are equivalent)
- Every statement must be terminated with a semicolon

```
1 if x = '0' and y = '0' or z = '1' then
2     blah; -- some useful statement
3     blah; -- some useful statement
4 end if;
5 if ( ((x = '0') and (y = '0')) or (z = '1') )
6     then
7     blah; -- some useful statement
8     blah; -- some useful statement
9 end if;
```

Listing 1. Example of parentheses that can improve clarity [4].

if, case, and loop statements have some rules you need to remember to save yourself time:

- Every if statement has a corresponding then component
- Each if statement is terminated with an end if;
- If you need an else if construct, the VHDL version is elsif
- Each case statement is terminated with an end case;
- Each loop statement has a corresponding end loop; statement

An **identifier** refers to the name given to various items in VHDL. Examples of identifiers in higher-level languages include variable names and function names. Examples of identifiers in VHDL include variable names, signal names and port names [4]. Here are some rules and recommendations:

- They should be self-describing
- They can be as long as you want
- Identifiers can only contain a combination of letters (A-Z and a-z), digits (0-9) and the underscore character ('_')
- Identifiers must start with an alphabetic character

⁸This section will be almost the same as chapter two of *Free Range VHDL* [4] but shortened. Most direct citations will be marked

- They must not end with an underscore and must not have two consecutive underscores
- Try to be consistent in your naming style

```

1 data_bus --descriptive name
2 WE --classic write enable
3 div_flag --real winner
4 port_A --provides some info
5 in_bus --input bus
6 clk --classic clock
7 clk_in
8 clk_out
9 mem_read_data

```

Listing 2. Valid identifiers [4].

```

1 3Bus_val -- begins with a number
2 DDD -- not self commenting
3 mid_$num -- illegal character
4 last__val-- consec. underscores
5 str_val_ -- ends with underscore
6 in -- uses VHDL reserved word
7 @#$$% -- total garbage
8 it_sucks -- try to avoid
9 Big_vAlUe-- valid but ugly
10 pa -- possibly lacks meaning
11 sim-val -- illegal character(dash)
12 DDE_SUX -- no comment

```

Listing 3. Invalid identifiers [4].

Some words are **reserved** in VHDL and cannot be used as identifiers. A partial list appears in Listing 4.

```

1 access after alias all attribute block
2 body buffer bus constant exit file
3 for function generic group in is
4 label loop mod new next null
5 of on open out range rem
6 return signal shared then to type
7 until use variable wait while with

```

Listing 4. Reserved Words [4].

VHDL code should always try to be as readable as possible. You should never try to memorize all these rules, when in doubt just check out a reference sheet like this one.

C. Blackbox Diagrams and Entities

Most often we don't care about how a design functions on the inside, like our logic gates we can abstract this. We will do this via the **black box approach** where we treat the circuits as boxes where we cannot look inside. We only know what inputs and what outputs it has. It reduces diagram complexity and allows us to reuse VHDL code (kind of like object-oriented programming in programming language).

VHDL provides the **entity** construct allowing us the abstract our circuits. The entity declaration describes our black box (Listing 5).

```

1 entity my_entity is
2 port (
3   port_name_1 : in std_logic ;
4   port_name_2 : out std_logic;
5   port_name_3 : inout std_logic ); --do not
6   forget the semicolon
7 end my_entity; -- do not forget this semicolon
8   either

```

Listing 5. The entity declaration in VHDL [4].

We introduce a lot of new things in Listing 5. The `port_name_x` is an identifier used to differentiate signals. The keyword after that (e.g. `in`) specifies the direction of the signal. `std_logic` refers to the data type of the signal (`std_logic` just means binary like we've used it before).

Drawing a black box diagram can help us write entity declarations with ease.

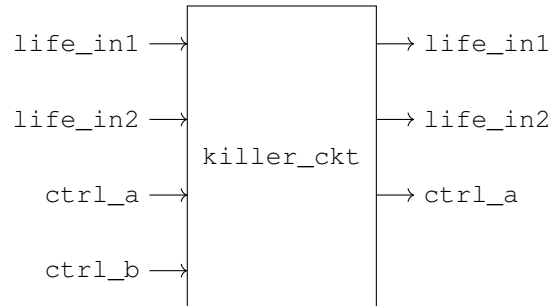


Fig. 11. Blackbox diagram for killer_ckt.

```

1 -----
2 -- interface description --
3 -- of killer_ckt --
4 -----
5 entity killer_ckt is
6 port (
7   life_in1 : in std_logic;
8   life_in2 : in std_logic;
9   ctrl_a, ctrl_b : in std_logic;
10  kill_a : out std_logic;
11  kill_b, kill_c : out std_logic);
12 end killer_ckt;

```

Listing 6. VHDL entity declaration for killer_ckt [4].

In Listing 6 you can see that we use a new syntax of `signal_a, signal_b : out std_logic;` to create two signals in just one line. Keep in mind that this should be used to make the code more readable, not to make it unreadably compact.

```

1 -----
2 -- Unlike the other examples, this is actually
3 -- an interface
4 -- for a MUX that selects one of four bus lines
5 -- for the output.
6 -----
7 entity mux4 is
8 port ( a_data : in std_logic_vector(0 to 7);
9       b_data : in std_logic_vector(0 to 7);
10      c_data : in std_logic_vector(0 to 7);
11      d_data : in std_logic_vector(0 to 7);
12      sel1, sel0 : in std_logic;
13      data_out : out std_logic_vector(7 downto 0));
14 end mux4;

```

Listing 7. VHDL entity declaration for mux4 [4].

In Listing 7 we use a something called a **bundle** to group signals together. We accomplish this by using `std_logic_vector` instead of `std_logic`. This is shown in Fig 12 with a dash through the signal with the number of signals it contains written above.

Lastly we need to talk about **VHDL standard libraries**. Even `std_logic` is not included automatically and needs to be imported. In Listing 8 you can see the standard libraries and how to import them.

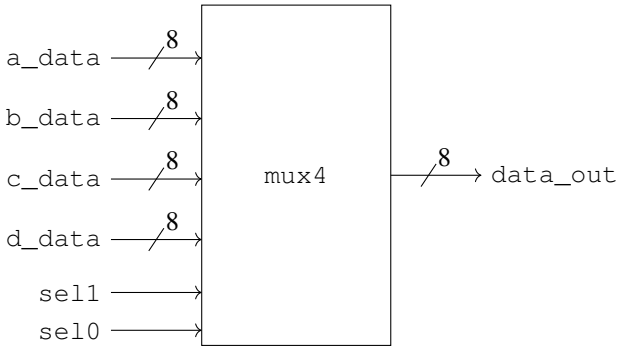


Fig. 12. Blackbox diagram for mux4.

```

1  -- library declaration
2  library IEEE;
3  use IEEE.std_logic_1164.all; -- basic IEEE
   library
4  use IEEE.numeric_std.all; -- IEEE library for
   the unsigned type and
5  -- various arithmetic operators
6
7  -- WARNING: in general try NOT to use the
   following libraries
8  -- because they are not IEEE standard libraries
9  -- use IEEE.std_logic_arith.all;
10 -- use IEEE.std_logic_unsigned.all;
11 -- use IEEE.std_logic_signed
12
13 -- entity
14 entity my_ent is
15 port ( A,B,C : in std_logic;
16       F : out std_logic);
17 end my_ent;
18 -- architecture
19 architecture my_arch of my_ent is
20     signal v1,v2 : std_logic_vector (3 downto 0);
21     signal u1 : unsigned (3 downto 0);
22     signal i1 : integer;
23 begin
24     u1 <= "1101";
25     i1 <= 13;
26     v1 <= std_logic_vector(u1); -- = "1101"
27     v2 <= std_logic_vector(to_unsigned(i1,
28                                   v2'length)); -- = "1101"
29
30     -- "4" could be used instead of "v2'length",
31     -- but the "length"
32     -- attribute makes life easier if you want
33     -- to change the size of v2
34
35     F <= NOT (A AND B AND C);
36 end my_arch;

```

Listing 8. Example on VHDL Standard Libraries [4].

III. COMBINATIONAL CIRCUITS

Combinational logic circuits are circuits where the output depends solely on the current combination of inputs (e.g., basic logic gates, multiplexers, and adders).

IV. SEQUENTIAL CIRCUITS

Unlike combinational circuits, sequential circuits have memory elements (e.g., flip-flops). Their outputs depend on both current inputs and previous states.

V. FINITE STATE MACHINES (FSM)

A **Finite State Machine** is a type of machine or automaton that has a finite set of states, which change based on the input and current state of the machine. It can be defined by a set of states, a next-state function, and an output function. The states need to be stored in flip-flops.

A. Moore Machines

One type of FSM is the **Moore Machine**. A Moore machine is a type of FSM, whose output only depends on its current state not on the inputs. We can model a *rising-edge detector* as a Moore machine (Fig. 13). Each state is shown as a circle with the state being labeled with S_0, S_1, S_2 . Each state has an associated output Y . Each state has arrows which either loop back to itself or connect it to other states. Each arrow has an input X associated with it which is the condition for the transition shown by that arrow. On the left, we can also see an arrow that shows in which state the FSM starts in.

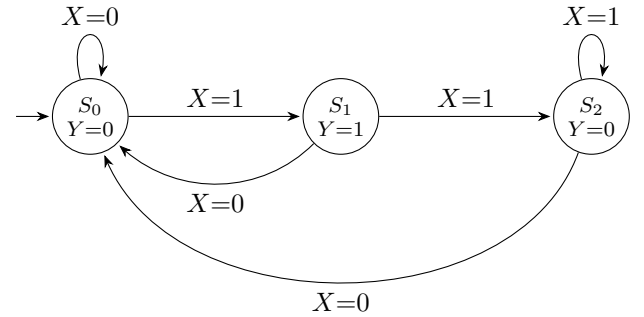
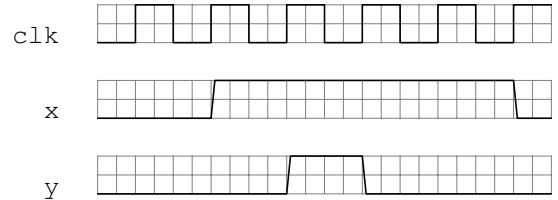
Fig. 13. Moore machine of a rising-edge detector. The output y is attached to the states.

Fig. 14. Simulated timing diagram matching GTKWave output for the Moore rising-edge detector.

The FSM in Figure 13 starts in S_0 outputting 0. If $X = 0$, we will stay in S_0 , however if we detect a rising edge in X , meaning that $X = 1$, the FSM transitions into S_1 and start outputting 1. If X now falls to 0 again, we transition back to S_0 , allowing us to detect another rising edge. If $X = 1$ while we're in S_1 , we transition to S_2 where the FSM outputs 0, because there is no rising edge anymore. Now if $X = 0$, we transition back to S_0 , because now there can be another rising edge. If it stays at $X = 1$ however, we will stay in S_2 and keep outputting 0, because staying at 1 doesn't constitute a rising edge.

In Figure 14 we can see the waveform generated by the code in Listing 9. We can see that X changes slightly after a rising edge. Because our states only transition on the rising

edge of the clock, the state only changes on the next clock cycle. We can see how Y only remains at 1 for one clock cycle and returns to 0 like we've specified in our state diagram.

```

1  library IEEE;
2  use IEEE.std_logic_1164.all;
3
4  -----
5  -- Rising Edge Detector --
6  -----
7  -- Detect a rising edge on X and --
8  -- outputs a 1 on Y, else Y = 0 --
9  -----
10
11 entity rising_edge_detector is
12     port ( X : in std_logic;
13           Y : out std_logic;
14           CLK : in std_logic);
15 end rising_edge_detector;
16
17 architecture arch_rising_edge_detector of
18     rising_edge_detector is
19     type state_type is (ST0, ST1, ST2);
20     signal PS, NS : state_type;
21 begin
22     -- On Clock rising edge, transition to next
23     state
24     sync_proc: process (CLK, NS)
25     begin
26         if (rising_edge(CLK)) then
27             PS <= NS;
28         end if;
29     end process sync_proc;
30
31     comb_proc: process (PS, X)
32     begin
33         Y <= '0'; -- pre-assign output
34         case PS is
35             when ST0 => -- State 0
36                 Y <= '0'; -- Moore Output
37                 -- Assign next state --
38                 if (X = '1') then NS <= ST1;
39                 else NS <= ST0;
40                 end if;
41             when ST1 => -- State 1
42                 Y <= '1'; -- Moore Output
43                 -- Assign next state --
44                 if (X = '1') then NS <= ST2;
45                 else NS <= ST0;
46                 end if;
47             when ST2 => -- State 0
48                 Y <= '0'; -- Moore Output
49                 -- Assign next state --
50                 if (X = '0') then NS <= ST0;
51                 else NS <= ST2;
52                 end if;
53             when others => -- catch-all condition
54                 -- It should never reach this
55                 -- anyways
56                 Y <= '0';
57                 NS <= ST0;
58         end case;
59     end process comb_proc;
60 end arch_rising_edge_detector;

```

Listing 9. Example VHDL solution for Figure 13

B. Mealy Machines

A **Mealy Machine** is a FSM whose output not only depends on its state, but also on its inputs. This often reduces the number of states. In our moore-type rising edge detector, we saw that the response was delayed because the output had to

wait for the state to transition (Fig. 14). We can remodel the rising edge detector as a mealy machine like shown in Figure 15.

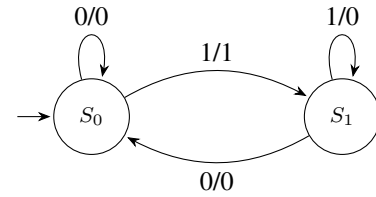


Fig. 15. Mealy machine of a rising-edge detector. Each arrow is annotated with X/Y , X being the input and Y the output

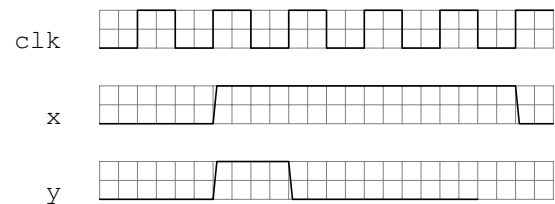


Fig. 16. Simulated timing diagram matching GTKWave output for the Mealy rising-edge detector.

We can see that we now have a faster reaction to the input. In Listing 10 there is an example of how you could implement Figure 15 in VHDL.

```

1  library IEEE;
2  use IEEE.std_logic_1164.all;
3
4  -----
5  -- Rising Edge Detector --
6  -----
7  -- Detect a rising edge on X and --
8  -----
9
10 entity rising_edge_detector is
11     port ( X : in std_logic;
12           Y : out std_logic;
13           CLK : in std_logic);
14 end rising_edge_detector;
15
16 architecture arc_rising_edge_detector of
17     rising_edge_detector is
18     type state_type is (ST0, ST1);
19     signal PS, NS : state_type;
20 begin
21     sync_proc : process (CLK, NS)
22     begin
23         if (rising_edge(CLK)) then
24             PS <= NS;
25         end if;
26     end process sync_proc;
27
28     comb_proc: process (PS, X)
29     begin
30         Y <= '0'; -- Output preassign
31         case PS is
32             when ST0 => -- State 0
33                 if (X = '1') then
34                     Y <= '1'; -- Change Output
35                     -- based on Input
36                     NS <= ST1;
37                 else
38                     Y <= '0'; -- Here other
39                     -- Output if X = '0'
40                     NS <= ST0;

```

```

38     end if;
39   when ST1 => -- State 1
40     if (X = '0') then
41       Y <= '0';
42       NS <= ST0;
43     else
44       Y <= '0';
45       NS <= ST1;
46     end if;
47   when others => -- Catch-All
48     Y <= '0';
49     NS <= ST0;
50   end case;
51 end process comb_proc;
52 end architecture;

```

Listing 10. Example VHDL solution for Figure 15

REFERENCES

- [1] Wiktionary contributors. circuit. Wiktionary, the free dictionary. [Online]. Available: <https://en.wiktionary.org/wiki/circuit>
- [2] ——. digital. Wiktionary, the free dictionary. [Online]. Available: <https://en.wiktionary.org/wiki/digital>
- [3] Wikipedia contributors, “Logic gate — Wikipedia, the free encyclopedia,” 2026, [Online; accessed 4-October-2026]. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Logic_gate&oldid=1377481077
- [4] B. Mealy and F. Tappero, *Free Range VHDL*. Free Range Factory, 2025.